

Git in VSCode — GUI Workflow Guide

A step-by-step guide to managing Git entirely from the VSCode interface.

1. What is Git?

Git is a distributed version control system used to track changes in code and collaborate with other developers. Nearly every professional software project today uses Git.

Developers use Git to:

- manage large codebases
- collaborate with teams
- review code via pull requests
- maintain production stability
- track the evolution of software

It is a core tool in modern software development workflows and the industry standard for software collaboration.

Think of Git as Google Docs version history for code — but much more powerful.

2. Why Developers Use Git

Git solves common problems in software development.

Problem	How Git Helps
Code breaks after changes	Revert to previous working version
Multiple developers editing files	Merge changes safely
Experimenting with new features	Use branches
Losing work	Full change history stored
Tracking who changed what	Commit history

3. Core Git Concepts

A. Repository (Repo)

A repository is a project folder that Git tracks.

It stores:

- files
- change history
- branches
- Commits

```
project/  
  app.js  
  index.html  
  style.css
```

B. Commit

A commit is a snapshot of your project at a specific moment.

Example commit message:

```
Add user login feature  
Fix bug in payment API  
Update navbar styling
```

Each commit includes:

- changes
- author
- timestamp
- commit message

4. Local vs Remote Repositories

Git is distributed, meaning every developer has a full copy locally. The remote repository stores the codebase in a central location.

Common remote platforms:

- GitHub
- GitLab
- Bitbucket

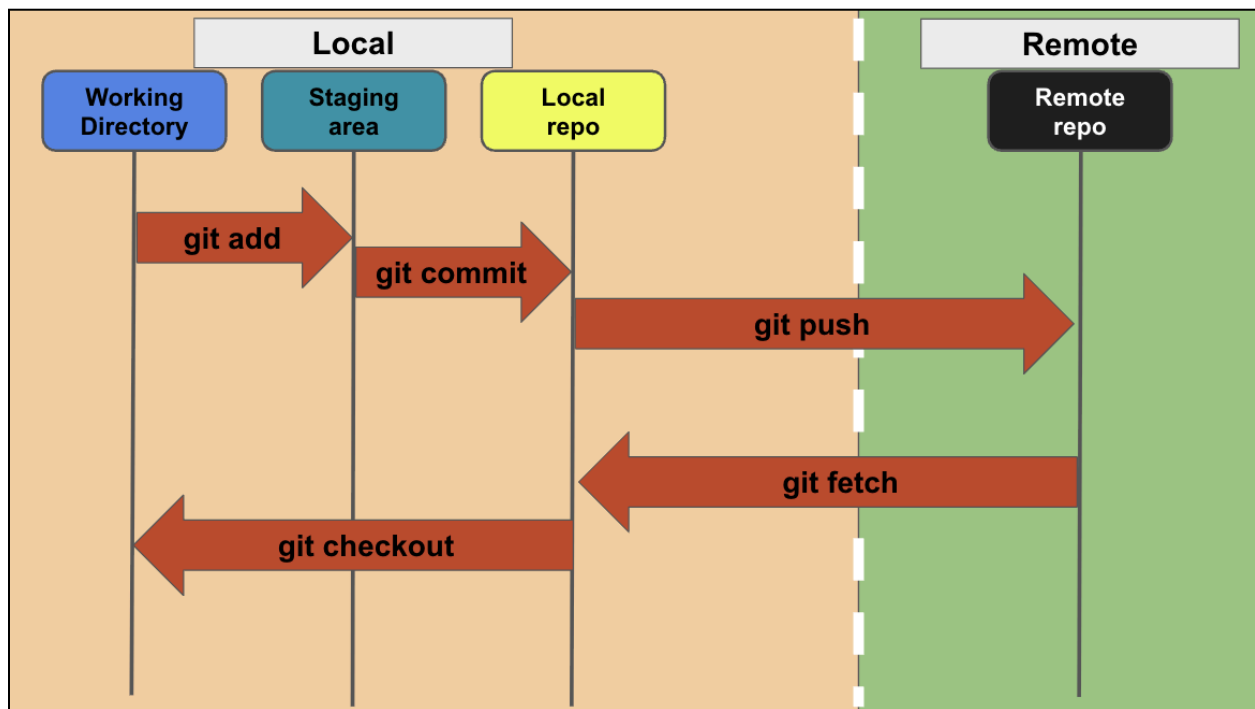
Developer Laptop (Local Repo)

↓ push

GitHub / GitLab (Remote Repo)

↑ pull

Other Developers



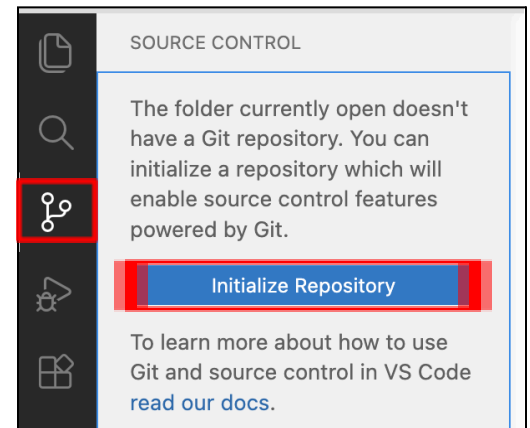
We will focus on creating a repository, adding files to staging and committing to the local repo for now.

5. Basic Git Workflow

A. Initialize a Repository

1. Open your project folder in VSCode via **File > Open Folder**
2. Open the **Source Control** panel:
Click the branch icon in the left Activity Bar, or Press **Ctrl+Shift+G** (Windows/Linux) / **Cmd+Shift+G** (Mac)
3. Click the **"Initialize Repository"** button

VSCode creates a .git folder in your project, enabling Git tracking. You'll now see all your files listed as untracked changes.

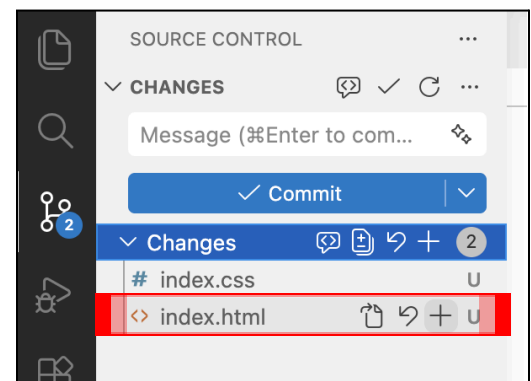


B. Stage Changes

After creating or editing files, they appear under **Changes** in the Source Control panel.

Stage a single file

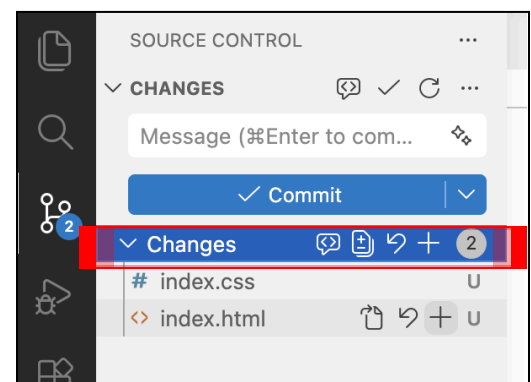
- Hover over a file in the **Changes** list
- Click the **+** (Stage Changes) icon that appears to the right



Stage all files at once

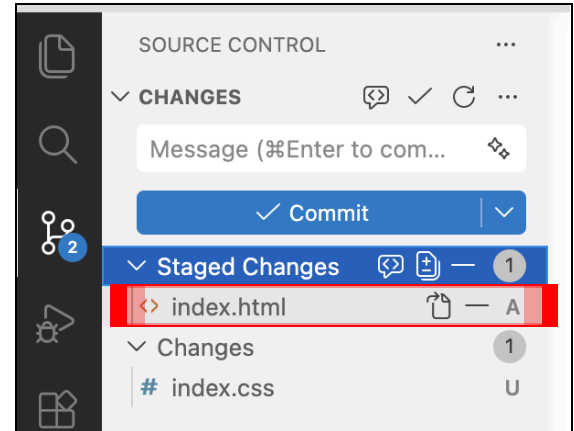
- Hover over the **Changes** heading
- Click the **+** icon next to the heading to stage everything.

Staged files move to a Staged Changes section, ready to be included in your next commit.



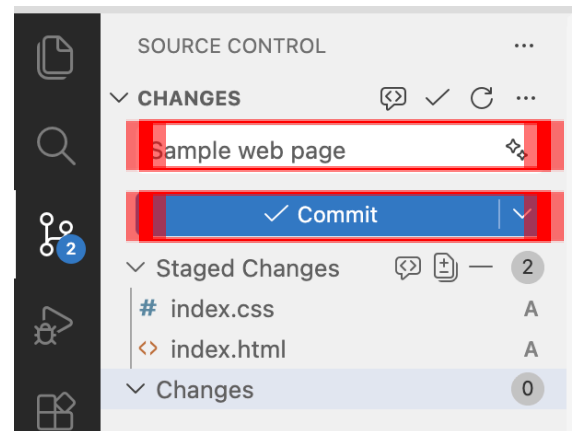
Unstage a file

Hover over a file in **Staged Changes**
Click the – (Unstage Changes) icon



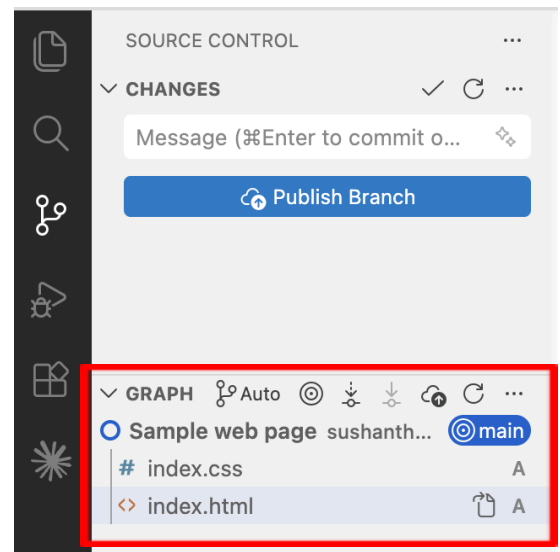
3. Commit Changes

1. At the top of the Source Control panel, click the **message box** (says "Message (Ctrl+Enter to commit)")
2. Type a descriptive commit message (e.g., Initial commit or Add homepage layout)
3. Commit by either:
Clicking the ✓ **Commit** button above the message box, or Pressing **Ctrl+Enter** (Windows/Linux) / **Cmd+Enter** (Mac)



Your changes are now saved as a commit in your local Git history.

On your first commit, VSCode sets up the default branch (usually main).



6. Some Important Git Commands (Optional, we will work with the GUI tools on VS Code)

Command	Purpose
<code>git init</code>	Initialize a repository
<code>git clone</code>	Copy a repo from remote
<code>git add</code>	Stage changes
<code>git commit</code>	Save a snapshot
<code>git push</code>	Upload changes
<code>git pull</code>	Download updates
<code>git branch</code>	Manage branches
<code>git merge</code>	Combine branches

7. Further Resources

1. <https://git-scm.com/learn>
2. <https://docs.github.com/en/get-started/using-git>